

CSS Module 1: Foundations

Selectors, the boilerplate, the box model, percentages, typography, backgrounds, borders and opacity.

8 topics

Contents

- 01 Selectors in CSS
- 02 CSS Boilerplate, Part 1
- 03 CSS Boilerplate, Part 2: box-sizing
- 04 The Percentage Unit and Parent Comparison
- 05 Text and Typography in CSS
- 06 Background Properties
- 07 Box Model Properties
- 08 Borders and Opacity

01 Selectors in CSS

Tag, class and id selectors · Grouping · Specificity basics

What a rule looks like

```
selector {  
  property: value; /* one declaration */  
}  
  
h1 { color: navy; font-size: 32px; }
```

The selector decides **which** elements are affected and the declaration block decides **what** happens to them.

The three basic selectors

Selector	Syntax	Selects
Tag (type)	<code>p { }</code>	Every paragraph on the page. Broad by nature, so it suits base styling.
Class	<code>.btn { }</code>	Every element carrying <code>class="btn"</code> . Reusable, and one element may carry several classes.
Id	<code>#header { }</code>	The single element carrying <code>id="header"</code> . An id must be unique on the page.

```
<button class="btn primary" id="checkout">Pay</button>  
  
button      { padding: 10px; } /* tag */  
.btn        { border-radius: 6px; }  
.primary    { background: #1f5f9e; }  
#checkout   { font-weight: 700; }
```

Combining selectors

Pattern	Meaning
<code>h1, h2, p</code>	Grouping. The same rules apply to all three.
<code>div p</code>	Descendant. Every paragraph anywhere inside a div.
<code>div > p</code>	Direct child only.
<code>p.note</code>	A paragraph that also has the class note. No space between them.
<code>*</code>	The universal selector, matching every element.

A SPACE CHANGES THE MEANING

`p .note` means an element with the class note *inside* a paragraph. `p.note` means the paragraph itself carries that class. This single space is one of the most common early mistakes.

Which rule wins

When several rules target the same element, the browser compares **specificity**. In increasing order of weight: tag selectors, then class selectors, then id selectors, and finally inline styles. If two rules have equal specificity, the one written later in the stylesheet wins.

Selector	Weight
<code>p</code>	0, 0, 1
<code>.card</code>	0, 1, 0
<code>#main</code>	1, 0, 0
<code>#main .card p</code>	1, 1, 1

PRACTICAL GUIDANCE

Build styling on classes. Ids are so specific that overriding them later becomes painful, and `!important` should be treated as a last resort rather than a tool, because it removes the normal cascade from the picture.

QUICK RECAP

- Tag selectors are broad, class selectors are reusable, id selectors are unique.
- A space between selectors means descendant; no space means the same element.
- Id beats class, class beats tag, and a later rule beats an earlier one of equal weight.

02 CSS Boilerplate, Part 1

Linking stylesheets · Browser defaults · A reliable starting file

Three ways to attach CSS

Method	Syntax	When to use
Inline	<code><p style="color:red"></code>	Almost never. It cannot be reused and it carries the highest specificity, so it is difficult to override.
Internal	<code><style> ... </style></code> in the head	Small demonstrations and single-file experiments.
External	<code><link rel="stylesheet" href="style.css"></code>	The standard approach for real projects, because the file is cached and shared across pages.

Why a boilerplate is needed

Every browser ships with a default stylesheet. That is why an `h1` is already large and bold and a `<body>` already has a margin. These defaults differ slightly between browsers, so a project normally begins by neutralising them.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html {
  font-size: 100%;          /* keeps 1rem equal to the user setting */
  scroll-behavior: smooth;
}

body {
  font-family: system-ui, "Segoe UI", Roboto, sans-serif;
  line-height: 1.5;
  color: #1c2431;
  background: #ffffff;
  -webkit-font-smoothing: antialiased;
}

img, video { max-width: 100%; display: block; }

a { color: inherit; text-decoration: none; }

button, input, select, textarea { font: inherit; }
```

What each line is actually doing

- `margin: 0; padding: 0;` removes inconsistent default spacing so that all spacing becomes a deliberate decision.
- `box-sizing: border-box` changes how width is calculated, which is covered in detail in the next topic.
- `img { max-width: 100% }` stops images from overflowing their container on small screens, and `display: block` removes the small gap caused by images being inline by default.
- `font: inherit` on form controls is essential, because buttons and inputs do not inherit the page font on their own.
- Setting `line-height` on the body gives every block of text comfortable spacing without repeating the property.

DO NOT CONFUSE A RESET WITH A DESIGN

A boilerplate removes inconsistencies. It should not contain colours, spacing scales or component styles. Keep it small, keep it at the top of the stylesheet, and never place project-specific rules inside the universal selector.

Order matters

Because rules of equal specificity are resolved by source order, the stylesheet is normally arranged as reset, then base element styles, then layout, then components, then utilities. Loading the boilerplate after component styles would silently undo them.

QUICK RECAP

- External stylesheets are the default choice; inline styles are the exception.
- Browsers apply their own default stylesheet, which the boilerplate neutralises.
- Keep the reset free of design decisions and place it first in the file.

03 CSS Boilerplate, Part 2: box-sizing

content-box against border-box · Why the reset exists

The problem this solves

By default every element uses `box-sizing: content-box`. In that mode the `width` you declare applies to the **content only**, and padding and border are added on top of it.

```
.card {
  width: 300px;
  padding: 20px;
  border: 5px solid #ccc;
}
/* content-box: rendered width = 300 + 20 + 20 + 5 + 5 = 350px */
```

The element therefore occupies fifty pixels more than the figure written in the code. In a layout where two cards are meant to sit side by side at fifty per cent each, this overflow is exactly what pushes the second card onto the next line.

The fix

```
* { box-sizing: border-box; }

.card {
  width: 300px;
  padding: 20px;
  border: 5px solid #ccc;
}
/* border-box: rendered width stays 300px, and the content area shrinks to 250px */
```

Value	width applies to	Result
<code>content-box</code>	Content only	Padding and border increase the visible size.
<code>border-box</code>	Content, padding and border together	The declared width is the width you see.

Two details worth knowing

- **Margin is never included** in either mode. Margin sits outside the box in both cases, so two elements of fifty per cent width with any horizontal margin will still overflow.
- The universal selector alone does not always reach pseudo-elements in older code, so the widely used version of the reset is written as `*::before, *::after { box-sizing: border-box; }`.

The box model in one picture

```

+-----+-----+-----+-----+-----+
| +-----+-----+-----+-----+-----+ |
| | +-----+-----+-----+-----+ | | | | | | |
| | | +-----+-----+-----+-----+ | | |
| | | | content | | | | |
| | | +-----+-----+-----+-----+ | | |
| | +-----+-----+-----+-----+ | |
+-----+-----+-----+-----+-----+
  
```

Reading outwards from the centre: content, then padding, then border, then margin. Background colour and background images cover the content and padding areas and extend beneath the border, but never beneath the margin.

WHY THIS BELONGS IN THE BOILERPLATE

Border-box makes width predictable, which means percentage-based layouts, grids and flex items behave the way the code reads. This is the single most valuable line in a starter stylesheet.

QUICK RECAP

- content-box adds padding and border to the declared width; border-box includes them.
- Margin always sits outside the box, in both modes.
- Applying border-box globally is standard practice in every modern project.

04 The Percentage Unit and Parent Comparison

What a percentage is measured against · Height traps · Padding percentages

A percentage is always relative to something

A percentage on its own means nothing. The browser resolves it against a specific property of the **containing block**, which is usually the parent element.

Property	Resolved against
<code>width</code>	The width of the containing block.
<code>height</code>	The height of the containing block, provided that height is definite.
<code>padding</code> and <code>margin</code> , on every side	The width of the containing block, including top and bottom values.
<code>font-size</code>	The font size of the parent.
<code>top</code> , <code>left</code> , <code>right</code> , <code>bottom</code>	The width or height of the containing block, depending on the axis.

The classic height problem

```
<div class="parent">
  <div class="child">Why am I not tall?</div>
</div>

.parent { }           /* no height declared      */
.child { height: 50%; } /* fifty per cent of what? */
```

The parent has no definite height of its own, because it is sized by its content. Half of an unknown value cannot be computed, so the browser ignores the declaration and the child stays at its content height.

There are three usual solutions:

- Give the parent a definite height, for example `height: 400px` or `height: 100vh`.
- Use a viewport unit on the child instead, so that it no longer depends on the parent.
- Make the parent a flex or grid container, where stretching happens naturally without percentages.

The old chain of `html, body { height: 100% }` exists for the same reason: each percentage needs a resolved height above it, all the way up to the viewport.

The padding percentage surprise

```
.box {
  width: 400px;
  padding-top: 50%; /* 200px, taken from the WIDTH, not the height */
}
```

Vertical padding and margin are measured against width. This looks strange at first, but it is what makes the classic aspect-ratio trick possible, where a wrapper with `padding-top: 56.25%` holds a sixteen by nine video. In modern code the `aspect-ratio` property does the same job far more clearly.

WATCH THE TOTAL

With `content-box`, three children at `width: 33.33%` with any padding will overflow the parent. With `border-box` they fit, but adding a horizontal margin will still push them over, because margin is never included in the width calculation.

QUICK RECAP

- Percentages resolve against the containing block, not against the viewport.
- A percentage height needs a definite height on the parent.
- Vertical padding and margin percentages are calculated from width.

05 Text and Typography in CSS

Font properties · Spacing · Alignment and overflow

Font properties

Property	Purpose
<code>font-family</code>	A prioritised list ending in a generic family, for example <code>"Inter", system-ui, sans-serif</code> . The browser uses the first font it can load.
<code>font-size</code>	Best expressed in <code>rem</code> so that the user's browser setting is respected.
<code>font-weight</code>	From 100 to 900, where 400 is normal and 700 is bold. A weight renders correctly only if that weight of the font is actually loaded.
<code>font-style</code>	Normal or italic.
<code>line-height</code>	The height of each line box. A unitless value such as 1.5 is preferred, since it scales with the font size of each element.
<code>letter-spacing</code> , <code>word-spacing</code>	Tracking adjustments, useful for headings and uppercase labels.

```
body {
  font-family: "Inter", system-ui, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
}
```

WHY UNITLESS LINE-HEIGHT

`line-height: 1.5` is recalculated for every element from its own font size. `line-height: 24px` is inherited as a fixed value, so a large heading inside that container ends up with cramped, overlapping lines.

Alignment, decoration and transformation

- `text-align` takes `left`, `right`, `center` or `justify`. It aligns inline content inside a block, and it does not move the block itself.
- `text-decoration` controls underlines and strikethroughs, and now accepts colour, style and thickness in the shorthand.
- `text-transform` changes the visual case to `uppercase`, `lowercase` or `capitalize`, while the underlying text in the HTML remains unchanged. This matters, because copy and paste returns the original text.
- `text-indent` indents the first line, and `vertical-align` aligns inline elements relative to the line box.

Handling long text

```
.title {
  white-space: nowrap;      /* prevent wrapping          */
  overflow: hidden;        /* hide what does not fit */
  text-overflow: ellipsis; /* show the trailing dots */
}

.para { word-break: break-word; } /* break very long unbroken strings */
```

The ellipsis only appears when all three properties are present together, which is why this combination is worth memorising.

Loading a web font

```
@import url("https://fonts.googleapis.com/css2?family=Inter:wght@400;700&display=swap");
```

A `<link>` tag in the HTML is preferred over `@import`, because an import blocks the stylesheet from loading in parallel. The `display=swap` setting shows a fallback font immediately instead of leaving invisible text while the font downloads.

QUICK RECAP

- Use rem for font sizes and a unitless value for line height.
- text-transform changes appearance only, never the underlying content.
- An ellipsis requires nowrap, hidden overflow and text-overflow together.

06 Background Properties

Colour and images · Repeat, position and size · Gradients

The individual properties

Property	Common values
<code>background-color</code>	Any colour, including <code>transparent</code> and <code>rgba()</code> .
<code>background-image</code>	<code>url("bg.jpg")</code> , or a gradient function. Several images may be listed, separated by commas.
<code>background-repeat</code>	<code>repeat</code> , <code>no-repeat</code> , <code>repeat-x</code> , <code>repeat-y</code> , <code>space</code> , <code>round</code> .
<code>background-position</code>	Keywords such as <code>center</code> , or a pair of values such as <code>50% 20%</code> or <code>20px 40px</code> .
<code>background-size</code>	<code>auto</code> , <code>cover</code> , <code>contain</code> , or explicit values.
<code>background-attachment</code>	<code>scroll</code> , <code>fixed</code> or <code>local</code> .
<code>background-clip</code> , <code>background-origin</code>	Decide whether the background stops at the padding box, the border box or the content box.

cover against contain

- `cover` scales the image until it fills the entire box. The aspect ratio is kept, so parts of the image are cropped. This is what a hero banner normally uses.
- `contain` scales the image until it fits completely inside the box. Nothing is cropped, but empty space may appear on two sides.

```
.hero {
  background-image: url("hero.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
```

The shorthand

```
.hero {
  background: #0f1723 url("hero.jpg") center / cover no-repeat;
}
/*      colour      image      position / size      repeat */
```

Within the shorthand the size must follow the position and be separated by a forward slash. Any property left out of the shorthand is reset to its default value, which is a frequent cause of a previously set background image disappearing.

Gradients

```
.card { background: linear-gradient(135deg, #1f5f9e, #16324f); }
.badge { background: radial-gradient(circle, #ffffff, #dfe6ef); }

/* a readable overlay above a photograph */
.banner {
  background:
    linear-gradient(rgba(0,0,0,.55), rgba(0,0,0,.55)),
    url("photo.jpg") center / cover;
}
```

A gradient is an image rather than a colour, so it belongs to `background-image`. When several backgrounds are listed, the **first one sits on top** and the last one sits at the bottom, which is exactly how the overlay above works.

POINTS TO REMEMBER

A background image is decorative, so it carries no alternative text and should not be used for content images. `background-attachment: fixed` is unreliable on several mobile browsers. Backgrounds extend under the padding and border but never under the margin.

QUICK RECAP

- cover crops to fill, contain fits without cropping.
- In the shorthand, size follows position after a slash, and omitted parts reset.
- In a stack of backgrounds, the first layer is the topmost one.

07 Box Model Properties

Margin, padding and dimensions · Shorthands · Margin collapse

Padding and margin

Padding is space **inside** the element, between the content and the border. It takes the background colour. Margin is space **outside** the element, always transparent, used to separate one element from another.

```
padding: 10px;           /* all four sides */
padding: 10px 20px;      /* vertical | horizontal */
padding: 10px 20px 15px; /* top | horizontal | bottom */
padding: 10px 20px 15px 5px; /* top | right | bottom | left, clockwise */
```

The same four patterns apply to `margin`, `border-width` and `border-radius`. Reading clockwise from the top is the reliable way to remember the four-value form.

Sizing properties

Property	Behaviour
<code>width</code> , <code>height</code>	A fixed or relative size for the box.
<code>min-width</code> , <code>min-height</code>	A floor that the element will not go below, even if the content is smaller.
<code>max-width</code> , <code>max-height</code>	A ceiling. <code>max-width</code> is the backbone of responsive containers.
<code>overflow</code>	<code>visible</code> , <code>hidden</code> , <code>scroll</code> or <code>auto</code> , deciding what happens to content that does not fit.

```
.container {
  width: 100%;
  max-width: 1200px;
  margin-inline: auto; /* equal automatic margins centre the block */
  padding-inline: 16px;
}
```

A width of one hundred per cent combined with a maximum width is the standard responsive container: it shrinks on small screens and stops growing on large ones.

Centring a block horizontally

`margin: 0 auto` works only when the element is a block-level box **with a definite width**. Without a width the box already fills the parent, so there is no free space for the automatic margins to divide. Note also that `auto` has no centring effect vertically.

Margin collapse

```
.first { margin-bottom: 30px; }
.second { margin-top: 20px; }
/* the gap between them is 30px, not 50px */
```

Adjacent vertical margins of block elements merge into one, and the larger value wins. Horizontal margins never collapse. Collapse is prevented by padding or a border between the elements, by an overflow value other than visible on the parent, or by using a flex or grid container, where margins never collapse at all. This is a strong reason to prefer `gap` for spacing inside modern layouts.

COMMON MISTAKES

Using margin where padding is required, so the clickable area of a button stays small · expecting `margin: auto` to centre vertically · giving an element both a width of one hundred per cent and horizontal padding under content-box, which causes overflow.

QUICK RECAP

- Padding grows the element from the inside; margin separates it from its neighbours.
- Shorthand values run clockwise from the top.
- Vertical margins collapse to the larger value, but never inside flex or grid.

08 Borders and Opacity

Border shorthand and radius · Outline · Opacity against alpha colours

Borders

```
border: 2px solid #1f5f9e;      /* width | style | colour */

border-top: 4px dashed red;
border-width: 1px 2px;
border-style: dotted;
border-color: transparent;
```

The `border-style` value is compulsory. A border declared with only a width and a colour will not appear at all, because the style defaults to `none`. Available styles include solid, dashed, dotted, double, groove and ridge.

Rounded corners

```
border-radius: 8px;              /* all four corners */
border-radius: 8px 0 8px 0;      /* clockwise from top-left */
border-radius: 50%;              /* a circle, when the box is square */
border-radius: 999px;            /* a pill-shaped button */
```

A percentage radius is calculated separately on each axis, which is why fifty per cent produces a perfect circle only when the width and height are equal; otherwise the result is an ellipse.

Outline is not border

Aspect	border	outline
Occupies layout space	Yes, it adds to the size of the box	No, it is drawn over the surroundings
Per-side control	Yes	No, it is drawn on all sides
Follows border-radius	Yes	Yes in current browsers

NEVER WRITE OUTLINE: NONE ON ITS OWN

The focus ring is how keyboard users know where they are on the page. If the default ring does not suit the design, replace it with a visible custom style through `:focus-visible` rather than removing it.

Opacity against alpha colours

```
.a { opacity: 0.5; }              /* the whole element, children included */
.b { background: rgba(31, 95, 158, 0.5); } /* only the background is translucent */
```

This is the distinction that matters in practice. `opacity` fades the element and everything inside it, so text placed on a faded card becomes hard to read. An alpha colour fades only the layer it is applied to, leaving the text at full strength. Modern syntax also allows `rgb(31 95 158 / 50%)` and hexadecimal values with an alpha pair, such as `#1f5f9e80`.

Related properties

- `visibility: hidden` hides the element but keeps its space, whereas `display: none` removes it from the layout entirely.
- An element with `opacity: 0` is invisible yet still clickable and still reachable by keyboard, which is a common accessibility bug.
- An opacity below one creates a new stacking context, which affects how `z-index` behaves inside that element.

QUICK RECAP

- A border needs a style, otherwise nothing is drawn.
- Outline does not take up space and must not be removed from focused elements.
- opacity affects the whole subtree; an alpha colour affects only one layer.